INDEX

Sr. No.	Date	Practical Title	Sign
1	14/12/2025	Demonstrate Matrix Multiplication & Eigen Decomposition using TensorFlow Library	
2	11/01/2026	Implementing Feed Forward Neural Network (FFNN) for Binary Classification on CSV Dataset	
3	11/01/2026	Implementing Multiclass Classification using Feed Forward Neural Network (FFNN)	
4	11/01/2026	Implementing Regression using Feed Forward Neural Network (FFNN)	
5	18/01/2026	Regularization Techniques in Deep Learning (L1, L2, L1 and L2)	
6	26/01/2026	Early Stopping in Neural Networks	
7	01/02/2026	Implementation of Convolutional Neural Network (CNN)	
8	01/03/2026	Implementation of Recurrent Neural Network (RNN)	
9	08/03/2026	Performing Encoding and Decoding of Images using Vanilla Autoencoder	

Practical 1

AIM: Demonstrate Matrix Multiplication & Eigen Decomposition using TensorFlow Library

CODE:

```
import keras

import tensorflow as tf

x=tf.constant([1,2,3,4,5,6],shape=[2,3])
x
```

```
<tf.Tensor: shape=(2, 3), dtype=int32, numpy=
array([[1, 2, 3],
       [4, 5, 6]], dtype=int32)>
```

```
y=tf.constant([7,8,9,10,11,12],shape=[3,2])
y
```

```
<tf.Tensor: shape=(3, 2), dtype=int32, numpy=
array([[ 7,  8],
       [ 9, 10],
       [11, 12]], dtype=int32)>
```

```
z=tf.matmul(x,y)
z
```

```
<tf.Tensor: shape=(2, 2), dtype=int32, numpy=
array([[ 58,  64],
       [139, 154]], dtype=int32)>
```

```
A=tf.random.uniform([2,2],minval=3,maxval=10,dtype=tf.float32)
A
```

```
<tf.Tensor: shape=(2, 2), dtype=float32, numpy=
array([[6.228286 , 6.569197 ],
       [4.7863626, 3.6327891]], dtype=float32)>
```

```
eigen_values,eigen_vectors = tf.linalg.eigh(A)

print(eigen_values)
```

```
tf.Tensor([-0.02863775  9.889713  ], shape=(2,), dtype=float32)
```

```
print(eigen_vectors)
```

```
tf.Tensor(
[[-0.6075828  0.7942564]
 [ 0.7942564  0.6075828]], shape=(2, 2), dtype=float32)
```

Practical 2

AIM: Implementing Feed Forward Neural Network (FFNN) for Binary Classification on CSV Dataset

CODE:

```
from keras.models import Sequential

from keras.layers import Dense

import pandas as pd

df=pd.read_csv('pima-indians-diabetes.csv',header=None)

df.sample(2)
```

	0	1	2	3	4	5	6	7	8
621	2	92	76	20	0	24.2	1.698	28	0
45	0	180	66	39	0	42.0	1.893	25	1

```
X=df.iloc[:,:-1]

df.shape
```

```
(768, 9)
```

```
X.shape
```

```
(768, 8)
```

```
y=df.iloc[:,-1]

y.shape
```

```
(768,)
```

```
from sklearn.model_selection import train_test_split

xtrain,xtest,ytrain,ytest=train_test_split(X,y,train_size=0.8,random_state
=30)

xtrain.shape,xtest.shape,ytrain.shape,ytest.shape
```

```
((614, 8), (154, 8), (614,), (154,))
```

```
nn=Sequential()

nn.add(Dense(input_dim=8,units=10,activation='relu'))
nn.add(Dense(units=1,activation='sigmoid'))

nn.compile(loss='binary_crossentropy',optimizer='adam',metrics=['accuracy'
])

nn.fit(xtrain,ytrain,batch_size=50,epochs=150)
```

```
Epoch 146/150
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.6992 - loss: 0.5837
Epoch 147/150
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.7168 - loss: 0.5522
Epoch 148/150
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.7180 - loss: 0.5466
Epoch 149/150
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.7286 - loss: 0.5600
Epoch 150/150
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.7304 - loss: 0.5349
[22]:

<keras.src.callbacks.history.History at 0x1f9c7e1d990>
```

```
predictions=nn.predict(xtest)
```

```
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step
```

```
Predictions
```

```
array([[2.19837055e-01],
       [4.77222890e-01],
       [7.14079216e-02],
       [4.91974264e-01],
       [4.42695588e-01],
       [3.62760499e-02],
       [8.04359913e-01],
```

```
import numpy as np
threshold=0.5
predicted_classes=(predictions>=threshold).astype(int)
```

```
predicted_classes
```

```
array([[0],
       [0],
       [0],
       [0],
       [0],
       [0],
```

```
from sklearn.metrics import
accuracy_score,confusion_matrix,classification_report
```

```
accuracy_score(ytest,predicted_classes)
```

```
0.7337662337662337
```

```
confusion_matrix(ytest,predicted_classes)
```

```
array([[87, 17],
       [24, 26]], dtype=int64)
```

```
print(classification_report(ytest,predicted_classes))
```

```
              precision    recall  f1-score   support

           0       0.78      0.84      0.81       104
           1       0.60      0.52      0.56        50

    accuracy                           0.73       154
   macro avg       0.69      0.68      0.68       154
weighted avg       0.73      0.73      0.73       154
```

Practical 3

AIM: Implementing Multiclass Classification using Feed Forward Neural Network (FFNN)

CODE:

```
from sklearn.datasets import load_iris

data=load_iris()

import pandas as pd

df=pd.DataFrame(data.data,columns=data.feature_names)

df.shape
```

```
(150, 4)
```

```
df.head(2)
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2

```
df['Class']=data.target

df.shape
```

```
(150, 5)
```

```
df.head(2)
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	Class
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0

```
X=df.iloc[:,:-1]

y=df.iloc[:,-1]

y
```

```
0      0
1      0
2      0
3      0
4      0
      ..
145    2
146    2
147    2
148    2
149    2
Name: Class, Length: 150, dtype: int64
```

```
import tensorflow as tf

encoded_y=tf.keras.utils.to_categorical(y)

encoded_y
```

```
array([[1., 0., 0.],
       [1., 0., 0.],
       [1., 0., 0.],
       [1., 0., 0.],
       [1., 0., 0.],
       [1., 0., 0.],
       [1., 0., 0.],
       [1., 0., 0.],
       [1., 0., 0.],
```

```
X.shape,y.shape
```

```
((150, 4), (150,))
```

```
from sklearn.model_selection import train_test_split

xtrain,xtest,ytrain,ytest=train_test_split(X,encoded_y,test_size=0.25,rand
om_state=42)

xtrain.shape,xtest.shape,ytrain.shape,ytest.shape
```

```
((112, 4), (38, 4), (112, 3), (38, 3))
```

```
from keras.models import Sequential

from keras.layers import Dense
```

```
model=Sequential()

model.add(Dense(units=8,input_dim=4,activation='relu'))
model.add(Dense(units=6,activation='relu'))
model.add(Dense(units=3,activation='softmax'))

model.compile(loss='categorical_crossentropy',optimizer='adam',metrics=['a
ccuracy'])

model.fit(xtrain,ytrain,epochs=150,batch_size=40)
```

```
Epoch 146/150
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.9821 - loss: 0.2653
Epoch 147/150
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - accuracy: 0.9821 - loss: 0.2630
Epoch 148/150
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.9821 - loss: 0.2606
Epoch 149/150
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.9821 - loss: 0.2584
Epoch 150/150
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - accuracy: 0.9821 - loss: 0.2563

<keras.src.callbacks.history.History at 0x2334d3eac50>
```

```
predictions=model.predict(xtest)
```

```
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 48ms/step
```

```
for i in range(5):
    print(predictions[i],ytest[i])
```

```
[0.02062118 0.65211344 0.32726535] [0. 1. 0.]
[0.9276105  0.00139662 0.07099293] [1. 0. 0.]
[7.03271304e-04 1.02859095e-01 8.96437645e-01] [0. 0. 1.]
[0.02222607 0.6531145  0.3246594 ] [0. 1. 0.]
[0.0179053 0.6672488 0.3148459] [0. 1. 0.]
```

Practical 4

AIM: Implementing Regression using Feed Forward Neural Network (FFNN)

CODE:

```
from keras.models import Sequential

from keras.layers import Dense

import pandas as pd

df = pd.read_csv('Advertising.csv')

df.shape
```

```
(200, 5)
```

```
df.head(2)
```

	ID	TV	Radio	Newspaper	Sales
0	1	230.1	37.8	69.2	22.1
1	2	44.5	39.3	45.1	10.4

```
df.columns
```

```
Index(['ID', 'TV', 'Radio', 'Newspaper', 'Sales'], dtype='object')
```

```
df.drop('ID',axis=1, inplace=True)

X = df.iloc[:,:-1]

X.shape
```

```
(200, 3)
```

```
y= df.iloc[:,-1]
```

```
y.shape
```

```
(200,)
```

```
y
```

```
0      22.1
1      10.4
2       9.3
3      18.5
4      12.9
       ...
195     7.6
196     9.7
197    12.8
198    25.5
199    13.4
Name: Sales, Length: 200, dtype: float64
```

```
from sklearn.model_selection import train_test_split

xtrai,xtest, ytrain,ytest = train_test_split(X,y , test_size =0.2,
random_state =42)

model =Sequential()

model.add(Dense(units=8,input_dim=3, activation='relu'))
model.add(Dense(units=6, activation='relu'))
model.add(Dense(units=1, activation='relu'))

model.compile(loss='mse', optimizer='adam', metrics=['mae'])

model.fit(xtrai,ytrain,epochs = 300,batch_size=40)
```

```
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 2.5221 - mae: 1.1841
Epoch 296/300
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 2.6127 - mae: 1.2321
Epoch 297/300
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 2.5696 - mae: 1.2410
Epoch 298/300
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 2.4208 - mae: 1.2032
Epoch 299/300
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 2.2602 - mae: 1.1859
Epoch 300/300
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 2.6306 - mae: 1.2814
[80]:

<keras.src.callbacks.history.History at 0x1f1d993e7a0>
```

```
from sklearn.metrics import r2_score, mean_absolute_error
```

```
predictions = model.predict(xtest)
```

```
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 54ms/step
```

```
r2_score(ytest, predictions)
```

```
0.8778809043031779
```

```
mean_absolute_error(ytest, predictions)
```

```
1.5845957124233248
```

Practical 5

AIM: Regularization Techniques in Deep Learning (L1, L2, L1 and L2)

CODE:

```
from sklearn.datasets import make_moons
from keras.models import Sequential
from keras.layers import Dense

X,y=make_moons(n_samples=100,noise=0.2,random_state=1)

X.shape
```

```
(100, 2)
```

```
y.shape
```

```
(100,)
```

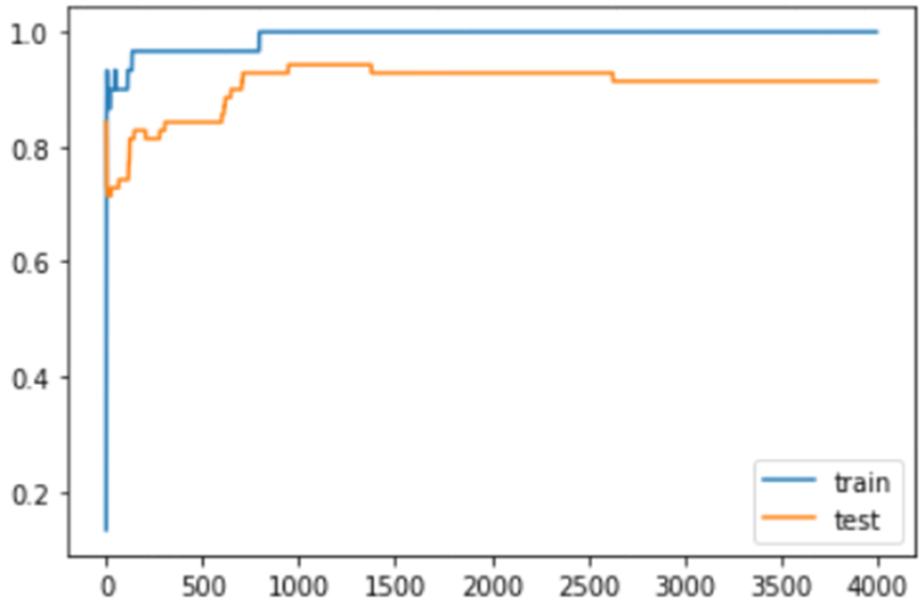
```
X_train,Xtest=X[:30,:],X[30:,:]

Y_train,Ytest=y[:30],y[30:]

model=Sequential()
model.add(Dense(500,input_dim=2,activation='relu'))
model.add(Dense(1,activation='sigmoid'))

model.compile(loss='binary_crossentropy',optimizer='adam',metrics=['accura
cy'])
```

```
history=model.fit(X_train,Y_train,validation_data=(Xtest,Ytest),epochs=400
0)
```

```
Epoch 3996/4000
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 83ms/step - accuracy: 1.0000 - loss: 2.3614e-04 - val_accuracy:
0.9143 - val_loss: 0.4839
Epoch 3997/4000
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 92ms/step - accuracy: 1.0000 - loss: 2.3595e-04 - val_accuracy:
0.9143 - val_loss: 0.4840
Epoch 3998/4000
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 84ms/step - accuracy: 1.0000 - loss: 2.3575e-04 - val_accuracy:
0.9143 - val_loss: 0.4840
Epoch 3999/4000
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 85ms/step - accuracy: 1.0000 - loss: 2.3555e-04 - val_accuracy:
0.9143 - val_loss: 0.4841
Epoch 4000/4000
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 85ms/step - accuracy: 1.0000 - loss: 2.3536e-04 - val_accuracy:
0.9143 - val_loss: 0.4841
```

```
from matplotlib import pyplot
pyplot.plot(history.history['accuracy'],label='train')
pyplot.plot(history.history['val_accuracy'],label='test')
pyplot.legend()
```

```
<matplotlib.legend.Legend at 0x205bfee9f00>
```



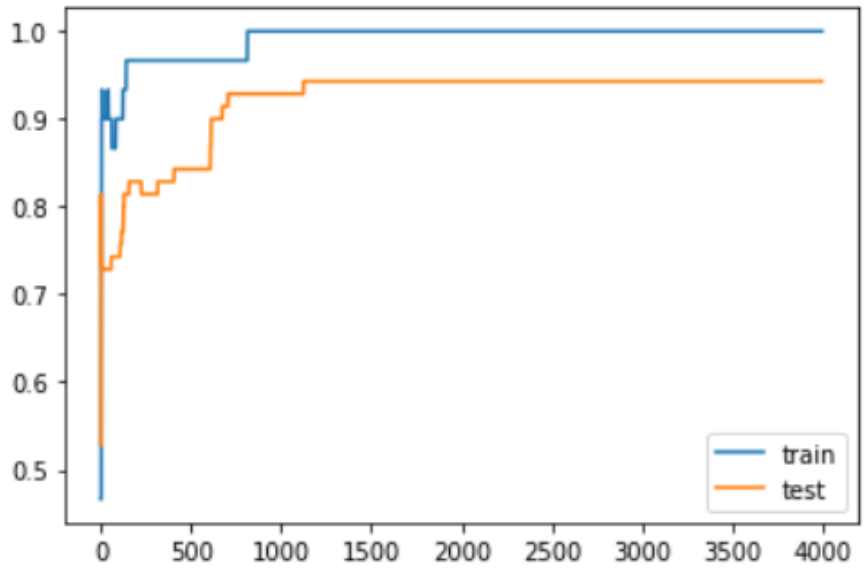
L2 Regularization

```
from keras.regularizers import l2
model_l2=Sequential()
model_l2.add(Dense(500,input_dim=2,activation='relu',kernel_regularizer=l2
(0.001)))
model_l2.add(Dense(1,activation='sigmoid'))

model_l2.compile(loss='binary_crossentropy',optimizer='adam',metrics=['acc
uracy'])

history2=model_l2.fit(X_train,Y_train,validation_data=(Xtest,Ytest),epochs
=4000)
```

```
Epoch 3996/4000
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 96ms/step - accuracy: 1.0000 - loss: 0.0160 - val_accuracy: 0.942
9 - val_loss: 0.2758
Epoch 3997/4000
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 85ms/step - accuracy: 1.0000 - loss: 0.0160 - val_accuracy: 0.942
9 - val_loss: 0.2758
Epoch 3998/4000
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 86ms/step - accuracy: 1.0000 - loss: 0.0160 - val_accuracy: 0.942
9 - val_loss: 0.2759
Epoch 3999/4000
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 78ms/step - accuracy: 1.0000 - loss: 0.0160 - val_accuracy: 0.942
9 - val_loss: 0.2760
Epoch 4000/4000
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 117ms/step - accuracy: 1.0000 - loss: 0.0160 - val_accuracy: 0.94
29 - val_loss: 0.2759
```

```
pyplot.plot(history2.history['accuracy'],label='train')
pyplot.plot(history2.history['val_accuracy'],label='test')
pyplot.legend()
```

```
<matplotlib.legend.Legend at 0x205cb099930>
```



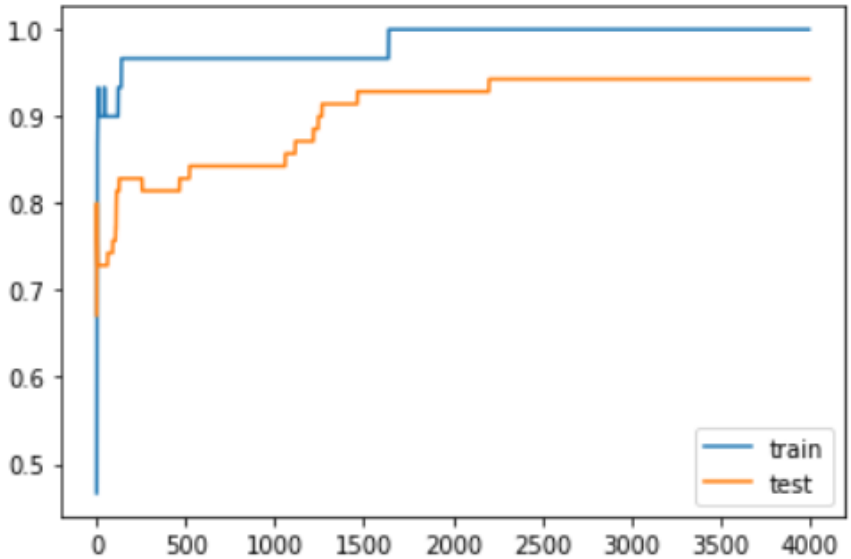
L1 Regularization

```
from keras.regularizers import l1
model_l1=Sequential()
model_l1.add(Dense(500,input_dim=2,activation='relu',kernel_regularizer=l1
(0.001)))
model_l1.add(Dense(1,activation='sigmoid'))

model_l1.compile(loss='binary_crossentropy',optimizer='adam',metrics=['acc
uracy'])

history3=model_l1.fit(X_train,Y_train,validation_data=(Xtest,Ytest),epochs
=4000)
```

```
Epoch 3995/4000
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 84ms/step - accuracy: 1.0000 - loss: 0.0284 - val_accuracy: 0.9429 - val_loss: 0.2314
Epoch 3996/4000
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 88ms/step - accuracy: 1.0000 - loss: 0.0284 - val_accuracy: 0.9429 - val_loss: 0.2314
Epoch 3997/4000
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 84ms/step - accuracy: 1.0000 - loss: 0.0284 - val_accuracy: 0.9429 - val_loss: 0.2315
Epoch 3998/4000
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 83ms/step - accuracy: 1.0000 - loss: 0.0284 - val_accuracy: 0.9429 - val_loss: 0.2315
Epoch 3999/4000
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 75ms/step - accuracy: 1.0000 - loss: 0.0284 - val_accuracy: 0.9429 - val_loss: 0.2315
Epoch 4000/4000
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 102ms/step - accuracy: 1.0000 - loss: 0.0284 - val_accuracy: 0.9429 - val_loss: 0.2315
```

```
pyplot.plot(history3.history['accuracy'],label='train')
pyplot.plot(history3.history['val_accuracy'],label='test')
pyplot.legend()
```

```
<matplotlib.legend.Legend at 0x205cee07130>
```



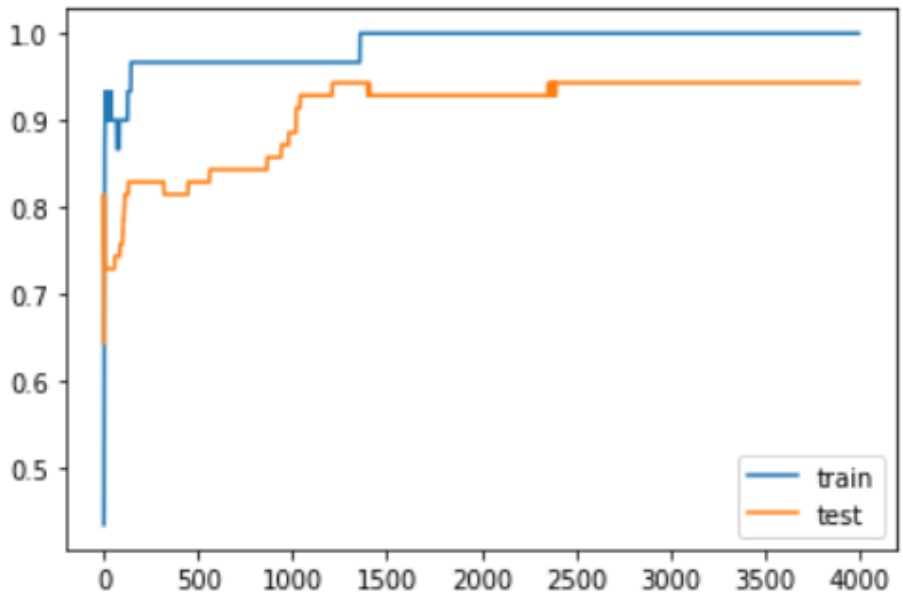
L1 and L2 Regularization

```
from keras.regularizers import l1_l2
model_l12=Sequential()
model_l12.add(Dense(500,input_dim=2,activation='relu',kernel_regularizer=l
1_l2(l1=0.001,l2=0.001)))
model_l12.add(Dense(1,activation='sigmoid'))

model_l12.compile(loss='binary_crossentropy',optimizer='adam',metrics=['ac
curacy'])

history4=model_l12.fit(X_train,Y_train,validation_data=(Xtest,Ytest),epoch
s=4000)
```

```
Epoch 3995/4000
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 86ms/step - accuracy: 1.0000 - loss: 0.0417 - val_accuracy: 0.9429 - val_loss: 0.2166
Epoch 3996/4000
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 85ms/step - accuracy: 1.0000 - loss: 0.0417 - val_accuracy: 0.9429 - val_loss: 0.2166
Epoch 3997/4000
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 87ms/step - accuracy: 1.0000 - loss: 0.0417 - val_accuracy: 0.9429 - val_loss: 0.2166
Epoch 3998/4000
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 83ms/step - accuracy: 1.0000 - loss: 0.0417 - val_accuracy: 0.9429 - val_loss: 0.2166
Epoch 3999/4000
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 160ms/step - accuracy: 1.0000 - loss: 0.0417 - val_accuracy: 0.9429 - val_loss: 0.2167
Epoch 4000/4000
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 93ms/step - accuracy: 1.0000 - loss: 0.0417 - val_accuracy: 0.9429 - val_loss: 0.2168
```

```
pyplot.plot(history4.history['accuracy'],label='train')
pyplot.plot(history4.history['val_accuracy'],label='test')
```

```
<matplotlib.legend.Legend at 0x205cae63fd0>
```



Practical 6

AIM: Early Stopping in Neural Networks

CODE:

```
import numpy as np
import pandas as pd
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
import matplotlib.pyplot as plt
from keras.datasets import fashion_mnist
from sklearn.model_selection import train_test_split
```

```
(train_images, train_labels), (test_images,
test_labels)=fashion_mnist.load_data()
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-label
s-idx1-ubyte.gz
29515/29515 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-image
s-idx3-ubyte.gz
26421880/26421880 ━━━━━━━━━━━━━━━━━━━━ 4s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-labels
-idx1-ubyte.gz
5148/5148 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images
-idx3-ubyte.gz
4422102/4422102 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
```

```
trn_images,valid_images,trn_labels,valid_labels=train_test_split(train_ima
ges,train_labels,test_size=0.3,random_state=2)
```

```
trn_images.shape
```

```
(42000, 28, 28)
```

```
trn_labels.shape
```

```
(42000,)
```

```
trn_images=trn_images/255.0
test_images=test_images/255.0
valid_images=valid_images/255.0
```

```
model1=keras.Sequential()
model1.add(layers.Flatten(input_shape=[28,28]))
model1.add(layers.Dense(300,activation='relu'))
model1.add(layers.Dense(10,activation='softmax'))

model2=keras.Sequential()
model2.add(layers.Flatten(input_shape=[28,28]))
model2.add(layers.Dense(300,activation='relu'))
model2.add(layers.Dense(10,activation='softmax'))

model1.compile(loss='sparse_categorical_crossentropy',optimizer='adam',met
rics=['accuracy'])

model2.compile(loss='sparse_categorical_crossentropy',optimizer='adam',met
rics=['accuracy'])

history=model1.fit(trn_images,trn_labels,epochs=50,validation_data=(valid_
images,valid_labels))
```

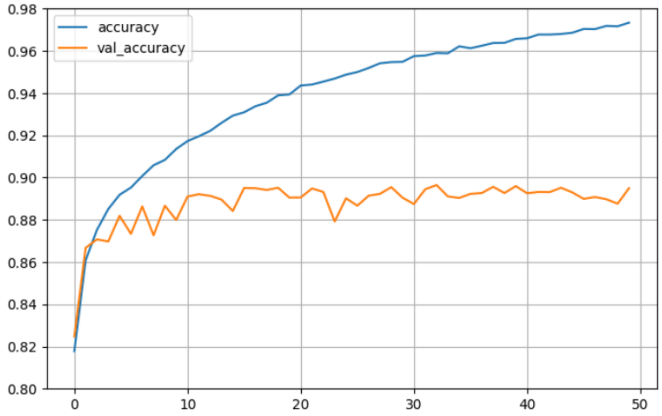
```
1313/1313 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9718 - loss: 0.0745 - val_accuracy:
0.8908 - val_loss: 0.4788
Epoch 48/50
1313/1313 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9733 - loss: 0.0720 - val_accuracy:
0.8897 - val_loss: 0.5057
Epoch 49/50
1313/1313 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9716 - loss: 0.0756 - val_accuracy:
0.8876 - val_loss: 0.5218
Epoch 50/50
1313/1313 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9735 - loss: 0.0701 - val_accuracy:
0.8949 - val_loss: 0.5014
```

```
model1.evaluate(test_images,test_labels)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 0s 805us/step - accuracy: 0.8909 - loss: 0.5802
```

```
[0.5827295780181885, 0.891700029373169]
```

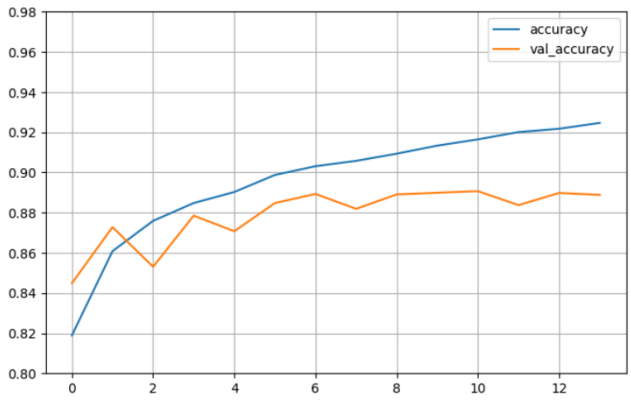
```
df=pd.DataFrame(history.history)
ind=[0,2]
df.iloc[:,ind].plot(figsize=(8,5))
plt.grid(True)
plt.gca().set_ylim(0.8,0.98)
plt.show()
```




```
callback=tf.keras.callbacks.EarlyStopping(patience=4,restore_best_weights=
True)

history1=model2.fit(trn_images,trn_labels,epochs=50,validation_data=(valid
_images,valid_labels),callbacks=[callback])
```

```
.313/1313 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9180 - loss: 0.2134 - val_accuracy:
).8907 - val_loss: 0.3120
:poch 12/50
.313/1313 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9203 - loss: 0.2121 - val_accuracy:
).8837 - val_loss: 0.3368
:poch 13/50
.313/1313 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9255 - loss: 0.2009 - val_accuracy:
).8898 - val_loss: 0.3219
:poch 14/50
.313/1313 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9264 - loss: 0.1929 - val_accuracy:
).8888 - val_loss: 0.3287
```

```
df=pd.DataFrame(history1.history)
ind=[0,2]
df.iloc[:,ind].plot(figsize=(8,5))
plt.grid(True)
plt.gca().set_ylim(0.8,0.98)
plt.show()
```



Practical 7

AIM: Implementation of Convolutional Neural Network (CNN)

CODE:

```
from keras.datasets import mnist

from keras.layers import Dense,Conv2D,Flatten,MaxPooling2D
from keras.models import Sequential

import matplotlib.pyplot as plt

(train_images,train_labels),(test_images,test_labels)=mnist.load_data()

train_images.shape
```

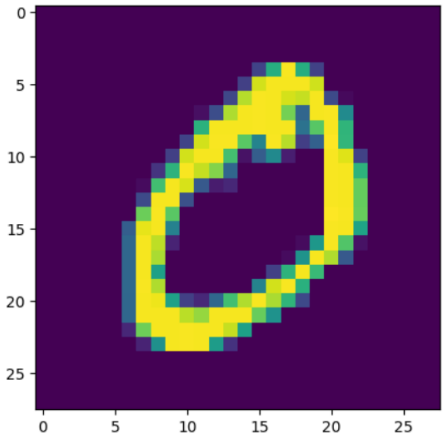
```
(60000, 28, 28)
```

```
test_images.shape
```

```
(10000, 28, 28)
```

```
plt.imshow(train_images[1])
```

```
<matplotlib.image.AxesImage at 0x1bc4ff5fd90>
```

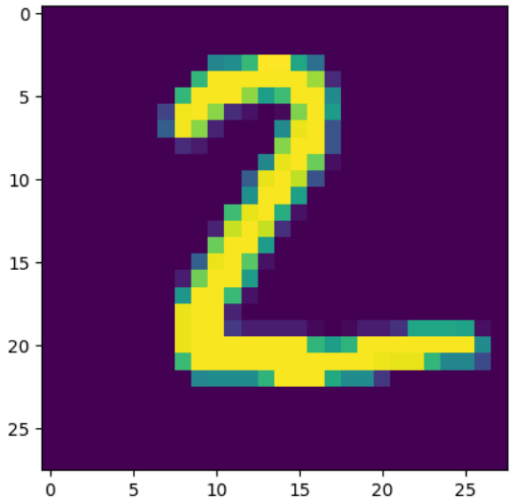



```
train_labels[1]
```

```
np.uint8(0)
```

```
plt.imshow(test_images[1])
```

```
<matplotlib.image.AxesImage at 0x1bc500154b0>
```




```
test_labels[1]
```

```
np.uint8(2)
```

```
train_images,test_images=train_images/255.0,test_images/255.0
```

```
import numpy as np
train_images=train_images.reshape(60000,28,28,1)
test_images=test_images.reshape(10000,28,28,1)
```

```
from keras.utils import to_categorical
```

```
train_labels=to_categorical(train_labels)
```

```
test_labels=to_categorical(test_labels)
```

```
test_labels[1]
```

```
array([0., 0., 1., 0., 0., 0., 0., 0., 0., 0.])
```

```
train_labels[1]
```

```
array([1., 0., 0., 0., 0., 0., 0., 0., 0., 0.])
```

```
cnnmodel=Sequential()
```

```
cnnmodel.add(Conv2D(32,(3,3),activation='relu',input_shape=(28,28,1)))
cnnmodel.add(MaxPooling2D(2,2))
cnnmodel.add(Conv2D(64,(3,3),activation='relu'))
cnnmodel.add(Flatten())
cnnmodel.add(Dense(10,activation='softmax'))
```

```
cnnmodel.compile(optimizer='adam',loss='categorical_crossentropy',metrics=
['accuracy'])
```

```
cnnmodel.fit(train_images,train_labels,epochs=3)
```

```
Epoch 1/3
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 13s 6ms/step - accuracy: 0.9605 - loss: 0.1343
Epoch 2/3
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 12s 7ms/step - accuracy: 0.9864 - loss: 0.0447
Epoch 3/3
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 12s 6ms/step - accuracy: 0.9906 - loss: 0.0309
[45]:

<keras.src.callbacks.history.History at 0x1bc4f511b70>
```

```
predictions=cnnmodel.predict(test_images)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step
```

```
for i in range(3):
    print(predictions[i])
    print(test_labels[i])
```

```
[1.8143362e-10 2.9068330e-09 2.1008157e-07 1.1171898e-06 1.0824721e-11
 1.8553825e-10 2.1170448e-16 9.9999869e-01 6.1540911e-10 3.6379500e-08]
[0. 0. 0. 0. 0. 0. 0. 1. 0. 0.]
[7.0841573e-09 1.6415630e-06 9.9996257e-01 1.9435864e-10 7.1634601e-11
 3.7622277e-11 3.5509656e-05 1.9725818e-11 2.3235026e-07 9.0565326e-11]
[0. 0. 1. 0. 0. 0. 0. 0. 0. 0.]
[3.0438819e-08 9.9995363e-01 1.6959082e-06 6.1261929e-08 1.2217726e-05
 1.1521748e-06 3.8150368e-08 1.7170161e-05 1.3944143e-05 6.5268821e-09]
[0. 1. 0. 0. 0. 0. 0. 0. 0. 0.]
```

Practical 8

AIM: Implementation of Recurrent Neural Network (RNN

CODE:

```
from keras.models import Sequential
from keras.layers import Dense
from keras.layers import LSTM
from keras.layers import Dropout

import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler

df=pd.read_csv("Google_Stock_Price_Train.csv")

df.head(3)
```

	Date	Open	High	Low	Close	Volume
0	1/3/2012	325.25	332.83	324.97	663.59	7,380,500
1	1/4/2012	331.27	333.87	329.08	666.45	5,749,400
2	1/5/2012	329.83	330.75	326.89	657.21	6,590,300

```
df.tail(3)
```

	Date	Open	High	Low	Close	Volume
1255	12/28/2016	793.70	794.23	783.20	785.05	1,153,800
1256	12/29/2016	783.33	785.93	778.92	782.79	744,300
1257	12/30/2016	782.75	782.78	770.41	771.82	1,770,000

```
training_df=df.iloc[:,1:2].values

training_df
```

```
array([[325.25],
       [331.27],
       [329.83],
       ...,
       [793.7 ],
       [783.33],
       [782.75]], shape=(1258, 1))
```

```
sc=MinMaxScaler(feature_range=(0,1))

sc_training_df=sc.fit_transform(training_df)

sc_training_df
```

```
array([[0.08581368],
       [0.09701243],
       [0.09433366],
       ...,
       [0.95725128],
       [0.93796041],
       [0.93688146]], shape=(1258, 1))
```

```
xtrain=[]
ytrain=[]

for i in range(60,1258):
    xtrain.append(sc_training_df[i-60:i,0])
    ytrain.append(sc_training_df[i,0])

xtrain=np.array(xtrain)

ytrain=np.array(ytrain)

xtrain[0:2]
```

```
array([[0.08581368, 0.09701243, 0.09433366, 0.09156187, 0.07984225,
        0.0643277 , 0.0585423 , 0.06568569, 0.06109085, 0.06639259,
        0.0614257 , 0.07474514, 0.02797827, 0.02379269, 0.02409033,
        0.0159238 , 0.01078949, 0.00967334, 0.01642607, 0.02100231,
        0.02280676, 0.02273235, 0.02810849, 0.03212665, 0.0433812 ,
        0.04475779, 0.04790163, 0.0440695 , 0.04648783, 0.04745517,
        0.04873875, 0.03936305, 0.04137213, 0.04034898, 0.04784582,
        0.04325099, 0.04356723, 0.04286033, 0.04602277, 0.05398467,
        0.05738894, 0.05714711, 0.05569611, 0.04421832, 0.04514845,
        0.04605997, 0.04412531, 0.03675869, 0.04486941, 0.05065481,
        0.05214302, 0.05612397, 0.05818885, 0.06540665, 0.06882953,
        0.07243843, 0.07993526, 0.07846566, 0.08034452, 0.08497656],
```

```
xtrain=np.reshape(xtrain,(xtrain.shape[0],xtrain.shape[1],1))

xtrain[0:2]
```

```
array([[[0.08581368],
        [0.09701243],
        [0.09433366],
        [0.09156187],
        [0.07984225],
        [0.0643277 ],
        [0.0585423 ],
        [0.06568569],
        [0.06109085],
        [0.06639259],
        [0.0614257 ],
        [0.07474514],
        [0.02797827],
        [0.02379269],
```

```
recurrent=Sequential()
recurrent.add(LSTM(units=50,return_sequences=True,input_shape=(xtrain.shap
e[1],1)))
recurrent.add(Dropout(0.2))
recurrent.add(LSTM(units=50,return_sequences=True))
recurrent.add(Dropout(0.2))
recurrent.add(LSTM(units=50,return_sequences=True))
recurrent.add(Dropout(0.2))
recurrent.add(LSTM(units=50))
recurrent.add(Dropout(0.2))
recurrent.add(Dense(units=1))

recurrent.compile(optimizer='adam',loss='mean_squared_error')

recurrent.fit(xtrain,ytrain,epochs=50,batch_size=32)
```

```
Epoch 46/50
38/38 ━━━━━━━━━━━━━━━━━━━━ 2s 40ms/step - loss: 0.0024
Epoch 47/50
38/38 ━━━━━━━━━━━━━━━━━━━━ 2s 40ms/step - loss: 0.0024
Epoch 48/50
38/38 ━━━━━━━━━━━━━━━━━━━━ 2s 40ms/step - loss: 0.0027
Epoch 49/50
38/38 ━━━━━━━━━━━━━━━━━━━━ 2s 40ms/step - loss: 0.0024
Epoch 50/50
38/38 ━━━━━━━━━━━━━━━━━━━━ 2s 42ms/step - loss: 0.0024

[24]:


<keras.src.callbacks.history.History at 0x221ec6729e0>
```

Practical 9

AIM: Performing Encoding and Decoding of Images using Vanilla Autoencoder

CODE:

```
import keras

from keras import layers

from keras.datasets import mnist

import numpy as np

encoding_dim=32

input_img=keras.Input(shape=(784,))
encoded=layers.Dense(encoding_dim,activation='relu')(input_img)
decoded=layers.Dense(784,activation='sigmoid')(encoded)

autoencoder= keras.Model(input_img,decoded)

encoder=keras.Model(input_img,encoded)

encoded_input=keras.Input(shape=(encoding_dim,))

decoder_layer=autoencoder.layers[-1]

decoder=keras.Model(encoded_input,decoder_layer(encoded_input))

autoencoder.compile(optimizer='adam',loss='binary_crossentropy')

(X_train,_),(X_test,_)=mnist.load_data()

X_train.shape
```

```
(60000, 28, 28)
```

```
X_test.shape
```

```
(10000, 28, 28)
```

```
X_train=X_train.astype('float32')/255.

X_test=X_test.astype('float32')/255.
```

```
X_train[0]
```

```
array([[0.        , 0.        , 0.        , 0.        , 0.        ,
        0.        , 0.        , 0.        , 0.        , 0.        ,
        0.        , 0.        , 0.        , 0.        , 0.        ,
        0.        , 0.        , 0.        , 0.        , 0.        ,
        0.        , 0.        , 0.        , 0.        , 0.        ,
        0.        , 0.        , 0.        ],
       [0.        , 0.        , 0.        , 0.        , 0.        ,
        0.        , 0.        , 0.        , 0.        , 0.        ,
        0.        , 0.        , 0.        , 0.        , 0.        ,
        0.        , 0.        , 0.        , 0.        , 0.        ,
        0.        , 0.        , 0.        , 0.        , 0.        ,
        0.        , 0.        , 0.        ],
```

```
X_train.shape[1:]
```

```
(28, 28)
```

```
X_train=X_train.reshape((len(X_train),np.prod(X_train.shape[1:])))

X_test=X_test.reshape((len(X_test),np.prod(X_test.shape[1:])))

X_train[0]
```

```
array([0.        , 0.        , 0.        , 0.        , 0.        ,
       0.        , 0.        , 0.        , 0.        , 0.        ,
       0.        , 0.        , 0.        , 0.        , 0.        ,
       0.        , 0.        , 0.        , 0.        , 0.        ,
       0.        , 0.        , 0.        , 0.        , 0.        ,
       0.        , 0.        , 0.        , 0.        , 0.        ,
       0.        , 0.        , 0.        , 0.        , 0.        ,
       0.        , 0.        , 0.        , 0.        , 0.        ,
       0.        , 0.        , 0.        , 0.        , 0.        ,
       0.        , 0.        , 0.        , 0.        , 0.        ,
       0.        , 0.        , 0.        , 0.        , 0.        ,
```

```
print(X_train.shape)
```

```
(60000, 784)
```

```
autoencoder.fit(X_train,X_train,epochs=50,batch_size=256,
                validation_data=(X_test,X_test))
```

```
235/235 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 0.0927 - val_loss: 0.0915
Epoch 46/50
235/235 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 0.0927 - val_loss: 0.0915
Epoch 47/50
235/235 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 0.0927 - val_loss: 0.0915
Epoch 48/50
235/235 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 0.0926 - val_loss: 0.0915
Epoch 49/50
235/235 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 0.0926 - val_loss: 0.0915
Epoch 50/50
235/235 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 0.0926 - val_loss: 0.0915

[75]:


<keras.src.callbacks.history.History at 0x1bf18bbaa90>
```

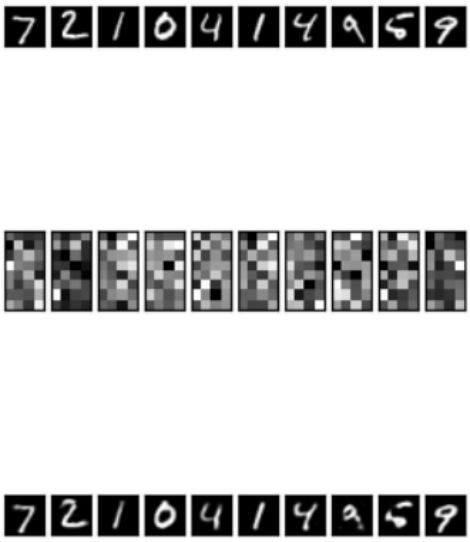
```
encoded_imgs=encoder.predict(X_test)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 0s 762us/step
```

```
decoded_imgs=decoder.predict(encoded_imgs)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 0s 741us/step
```

```
import matplotlib.pyplot as plt

n=10
plt.figure(figsize=(40,4))
```

```
<Figure size 4000x400 with 0 Axes>
<Figure size 4000x400 with 0 Axes>
```

```
for i in range (10):
    ax =plt.subplot(3,20,i+1)
    plt.imshow(X_test[i].reshape(28,28))
    decoded_imgs
    ax.get_xaxis().set_visible(False)
    ax.get_yaxis().set_visible(False)
    ax=plt.subplot(3,20,i+1+20)
    plt.imshow(encoded_imgs[i].reshape(8,4))
    plt.gray()
    ax.get_xaxis().set_visible(False)
    ax.get_yaxis().set_visible(False)
    ax=plt.subplot(3,20,2*20+i+1)
    plt.imshow(decoded_imgs[i].reshape(28,28))
    plt.gray()
    ax.get_xaxis().set_visible(False)
    ax.get_yaxis().set_visible(False)
plt.show()
```